

CMPT 276 - Assignment 4
Project Group 14
Team: Joshua Kim, Sarah Harb

Code Review

1. Code Smell Identified: Refused Bequest

In the GameObject class, the tick() method was originally defined as an abstract method. This forced all subclasses to implement it, even those that did not require tick(). As a result, some classes contained empty tick() methods. The tick() method was changed to a concrete method in the GameObject class, and all empty implementations of tick() were removed. Subclasses now only override tick() if they require custom behaviour.

Commit Reference: b7b147b

2. Code Smell Identified: Long Method

In the Game class, the constructor contained logic for creating and initializing all game objects, such as bushes, enemies and rewards. This made the constructor excessively large and difficult to maintain. The object creation logic was extracted into separate methods - addBushes() and initializeGameObjects(), which manages the setup process and delegates tasks to specific helper methods. The Game constructor was changed so that it only calls initializeGameObjects().

Commit Reference: 3f5eca8, ff4a50a

3. Code Smell Identified: Speculative Generality

The Cat class was initially created as a subclass of MovingEnemy (and GameObject, by extension) and included in the EnemyType enum. However, the class contained empty implementations of inherited methods and was never fully implemented or instantiated in the game. This indicates speculative generality, as the class and its enum entry were created in anticipation of future functionalities that never materialized. The Cat class and its entry CAT in the EnemyType enum were removed.

Commit Reference: b7b147b

4. Code Smell Identified: Confusing Class Hierarchy

The GameObject class had an abstract method render(), but its direct subclasses (i.e. Obstacle, Punishment, MovingEnemy, Reward) did not implement it. Instead, the responsibility was deferred to their own child classes. This created ambiguity in the hierarchy and made the design contract unclear. A public abstract void render() method was added to all non-implementing subclasses of GameObject. This clarified the responsibility for the render() implementation.

Commit Reference: b7b147b

5. Code Smell Identified: Long Method

In the original Doug class, the collision method was long and handled multiple responsibilities. It contained collision logic for various types of game objects, resulting in a single method that was difficult to read, maintain, and test. Thus, the collision method was

split into smaller, more focused methods. The `checkCollisionX` and `checkCollisionY` methods handle collision checks along the X and Y axes, respectively, and move object-specific collision logic to a helper method. The `handleCollision` method uses a switch statement to route collisions to appropriate handlers (`handleEnemyCollision`, `handleRewardCollision`, `handleExitCollision`, and `handleObstacle`). Then, we separated cases for each object type. By refactoring the collision method into these smaller methods, we improved readability and maintainability.

Commit Reference: b96e359

6. Code Smell Identified: Long Method

Another Long Method was identified in `Doug` class. The `tick` method was quite long and handled multiple responsibilities, such as predicting new positions, handling collisions, updating animation state, and clamping `Doug`'s position to the screen bounds. Thus, the `tick` method was refactored into smaller, focused methods. Firstly, `predictNewPosition` calculates `Doug`'s predicted position based on velocity and speed. Secondly, `handleCollision` checks and resolves collisions for the predicted position. Thirdly, `updateAnimationState` updates `Doug`'s movement state, facing direction, and animation frames. Lastly, `clampPosition` method ensures `Doug` remains within the screen bounds. These changes improved readability and maintainability by isolating distinct tasks into their respective methods, reducing the complexity of the `tick` method.

Commit Reference: e7e70af

7. Code Smell Identified: Hardcoded Values

In the `Game` class, various values such as window dimensions, file paths for resources (e.g., background images and music), and grid settings were hard coded directly in the class. This made the code less maintainable and more error-prone, as any changes required modifying multiple locations within the class. It also limited flexibility and reusability, particularly if different configurations were needed for testing or future expansions. A new `GameConfig` class was introduced to centralize all configuration constants. Hardcoded values, including window dimensions, block sizes, grid size, and file paths, were replaced with constants defined in `GameConfig`. The `Game` class was updated to reference these constants, ensuring all configuration-related changes can now be made in one place.

Commit Reference: ad483fd0

8. Code Smell Identified: Long Parameter List

In the `Game` class, the method `generateRandomObjects` had a long parameter list, which included multiple parameters like the count of objects, the object type, the handler, and the minimum distance from `Doug`. This made the method harder to read and maintain, as adding or modifying parameters required changes across all call sites, leading to increased complexity. To address this, a new class `GenerationConfig` was created to encapsulate these parameters. This reduced the method's signature to a single parameter, improving readability and maintainability. The `generateRandomObjects` method now accepts a `GenerationConfig` object that holds all relevant parameters.

Commit Reference: 77b4800